

Resource Allocation

Richard T. B. Ma

School of Computing

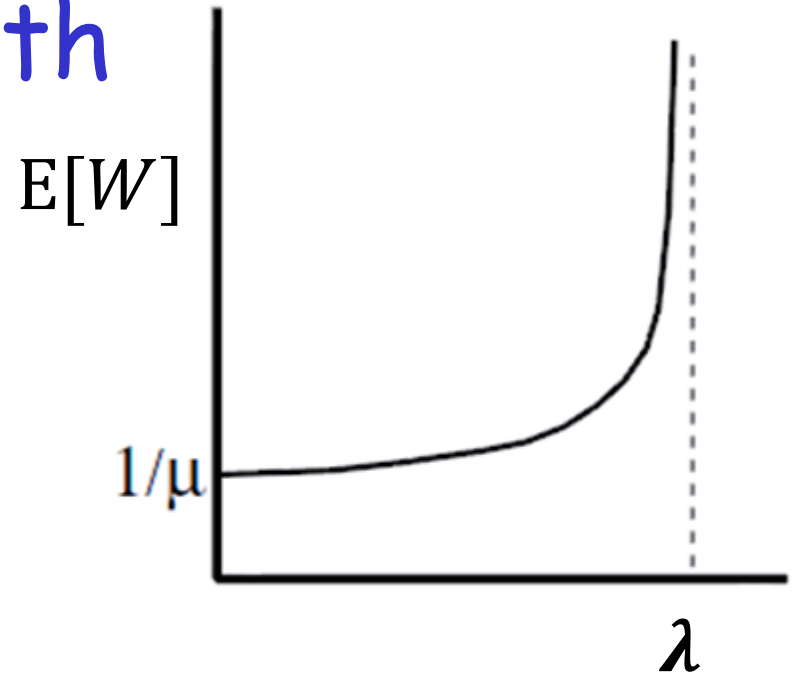
National University of Singapore

CS4226: Internet Architecture

Effective Bandwidth

□ What we have learned

❖ $E[W] = \frac{1}{\mu - \lambda}$



○ more capacity

- accommodates higher throughput
- provides lower delay

○ positive correlation of delay and throughput

○ given delay guarantee, achieved throughput:

$$\lambda = \mu - \frac{1}{E[W]}$$

Resource Allocation

□ two packet flows going through a link

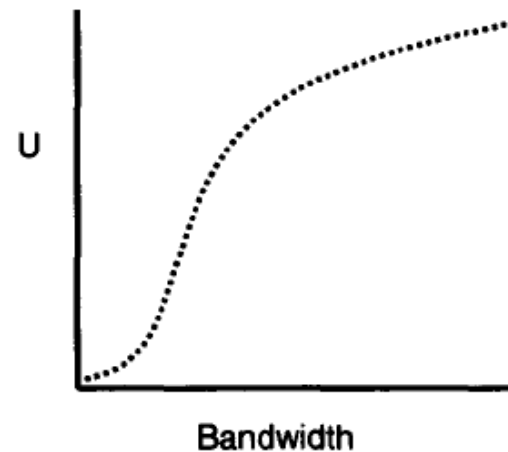
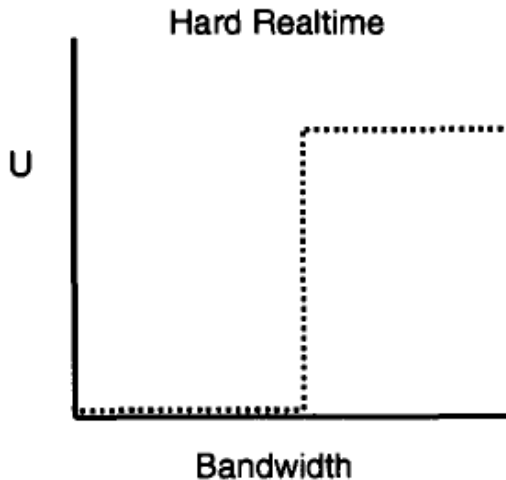
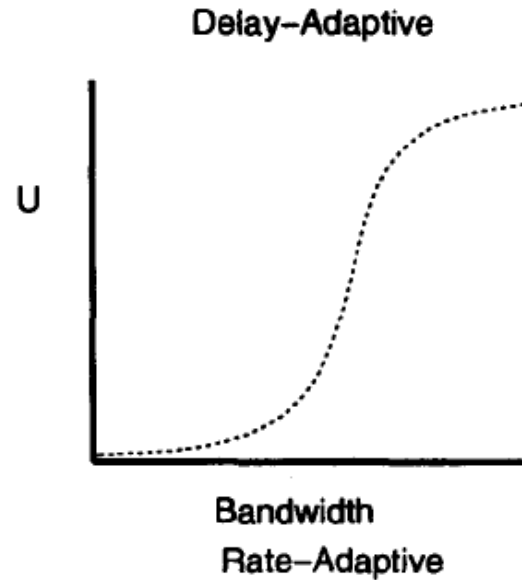
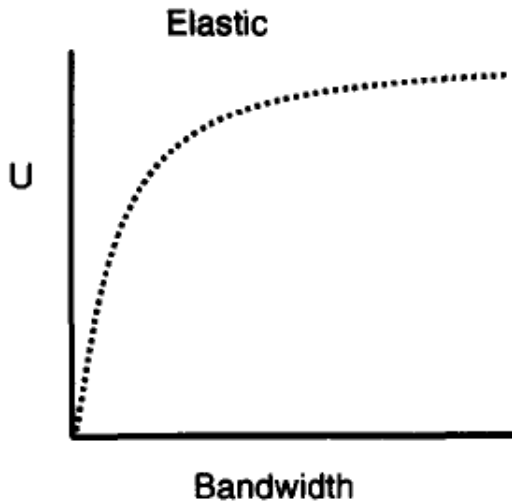
- flow 1 has a rate λ_1 packets/second
- flow 2 has a rate λ_2 packets/second
- the link as capacity μ packets/second
- what if we divide capacity into $\mu = \mu_1 + \mu_2$ and serve two flows separately
 - packets from flows might have different average delays
- what if we want to guarantee delays?
 - $\lambda_1 + \lambda_2$ has to be bounded

Why Do We Allocate Resource?

□ Internet provides best-effort service

- No guarantee for packet delivery
- Suitable for elastic traffic
 - Remote terminal (Telnet), file download (FTP), name service (DNS), email (SMTP)
- But not suitable for real-time applications
 - Voice and video, e.g. Skype, Zoom ...
- Solution: allocate different amount resource to different application flows
- A more general utility perspective:
 - User's happiness U_i as a function of performance metrics, e.g. delay or throughput

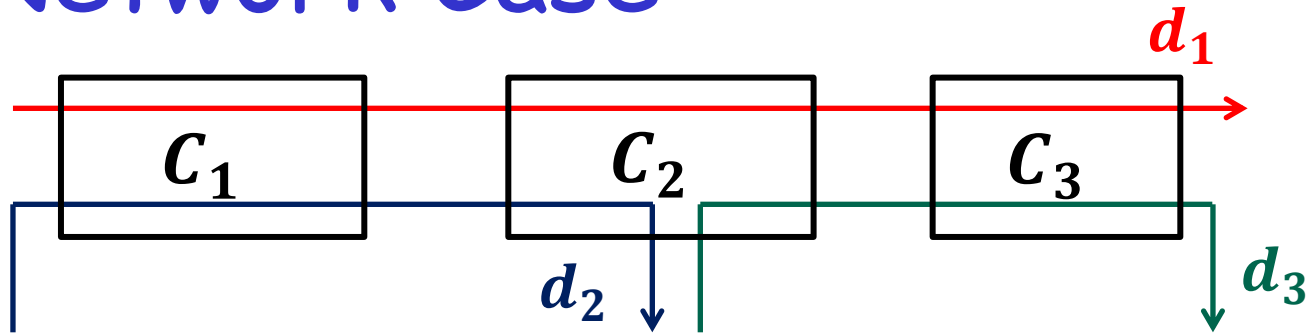
How Does Utility Look Like?



What is Fairness?

- Equal share of resources
- A single-resource with capacity $C = 10$
 - Demand $\mathbf{d} = (d_1, d_2, d_3, d_4) = (3, 4, 5, 6)$
 - $\mathbf{x} = (x_1, x_2, x_3, x_4) = (2.5, 2.5, 2.5, 2.5)$
 - Demand $\mathbf{d} = (d_1, d_2, d_3, d_4) = (1, 4, 5, 6)$
 - $\mathbf{x} = (x_1, x_2, x_3, x_4) = (1, 3, 3, 3)$
- Idea:
 - Users of small demand get all they want.
 - Users of large demand evenly split the rest.

A Network Case



□ Resource Capacity,

- For example, $C = (C_1, C_2, C_3) = (3, 6, 7)$

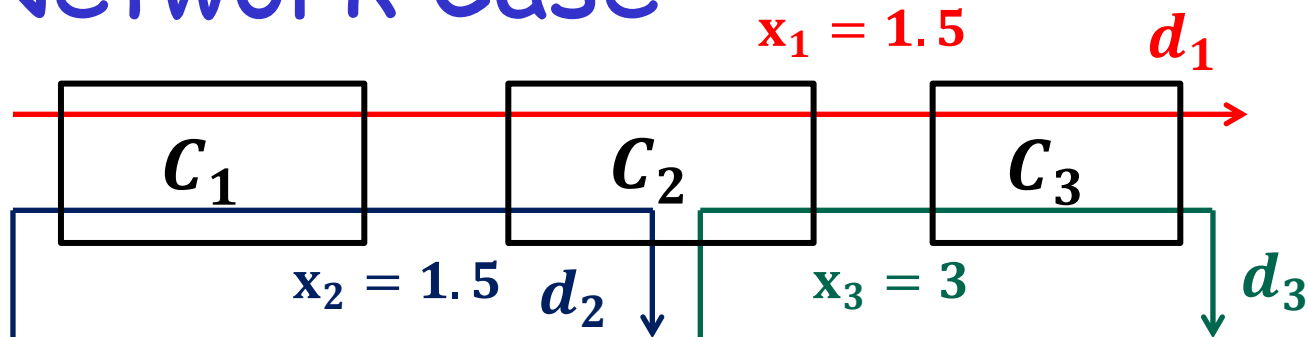
□ Flow Demand

- For example, $d = (d_1, d_2, d_3) = (2, 4, 4)$

❖ Feasible solution $x = (x_1, x_2, x_3)$ such that

- $0 \leq x_i \leq d_i$ for all i
- $x_1 + x_2 \leq C_1$; $x_1 + x_2 + x_3 \leq C_2$; $x_1 + x_3 \leq C_3$

A Network Case



- $C = (3, 6, 7)$ and $d = (2, 4, 4)$
- Resource C_3 is not a bottleneck
- Fair-share at C_2 is $x_1 = x_2 = x_3 = 2$
- Fair-share at C_1 is $x_1 = x_2 = 1.5$
$$x_1 = x_2 = 1.5; x_3 = C_2 - x_1 - x_2 = 3$$
- ❖ This is called a max-min fair solution!

Max-Min Fairness

- A feasible allocation is max-min fair if and only if an increase of any rate within the feasible domain must be at the cost of a decrease of an already smaller or equal rate.
- x is max-min fair if for any feasible y , if $y_i > x_i$, then $\exists j$ such that $y_j < x_j \leq x_i$.
- Always exists a max-min fair solution?
- If so, is it unique?
- How to obtain it?

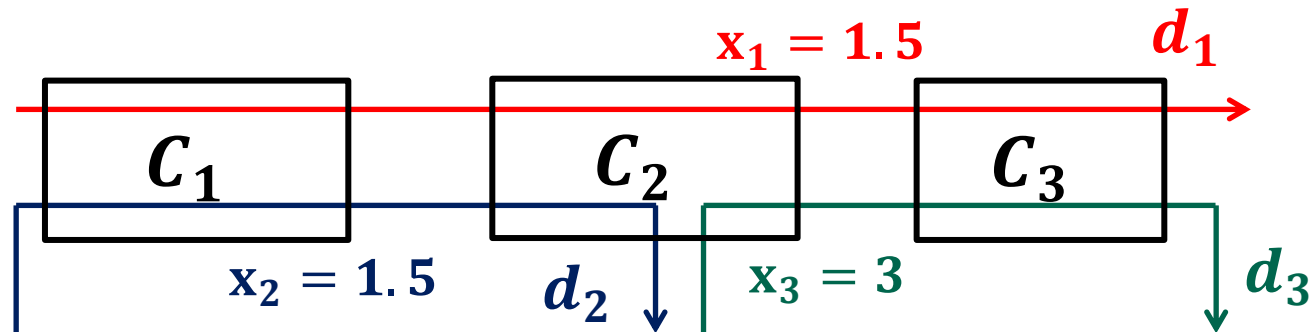
References

- ❖ Bozidar Radunovic and Jean-Yves Le Boudec, "A unified framework for max-min and min-max fairness with applications", IEEE/ACM Transactions on Networking, Vol. 15, No. 5, pp. 1073-1083, October, 2007.
- ❖ Jean-Yves Le Boudec, "Rate Adaptation, Congestion Control and Fairness: A Tutorial", 2000.

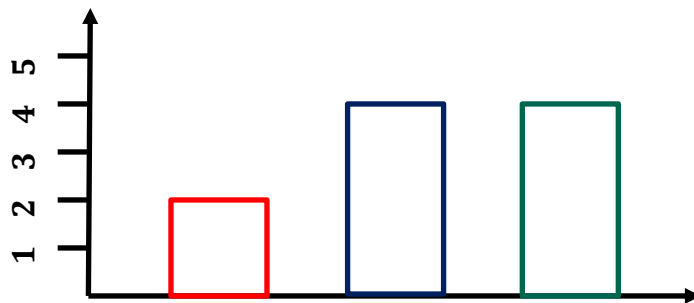
Bottleneck Resource

- In the network case, we say that resource r is a bottleneck for flow i if and only if
 - resource r is saturated and
 - flow i has the maximum rate among all flows using resource r .
 - ❖ flow i cannot get more resource r if allocation is fair; otherwise, it hurts flows with lower rates
- Theorem: When each flow has an infinity demand under a network system, a flow allocation is max-min fair if and only if every flow has a bottleneck resource.

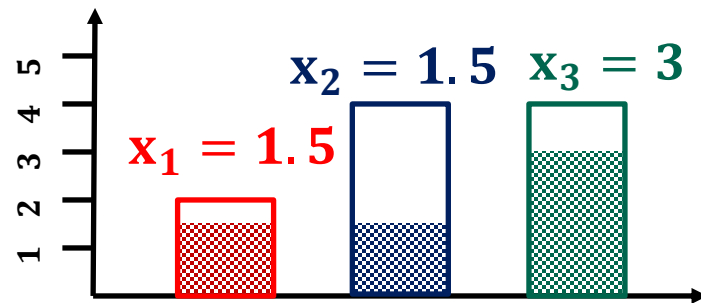
Water Filling Algorithm



□ $C = (3, 6, 7)$ and $d = (2, 4, 4)$

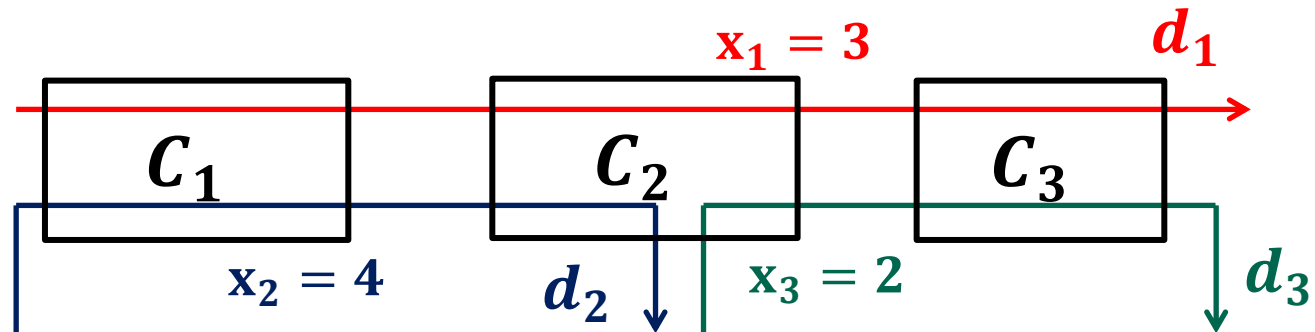


□ List the demands as empty buckets

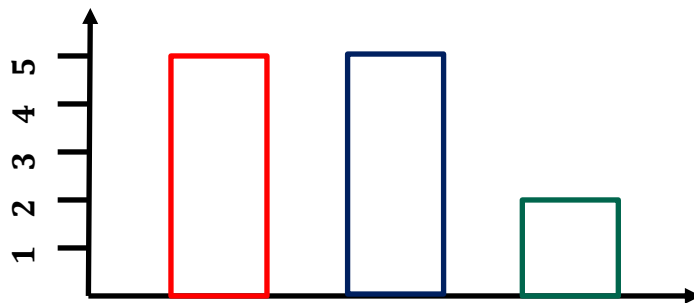


□ "Fill-in water" until one hits bottleneck

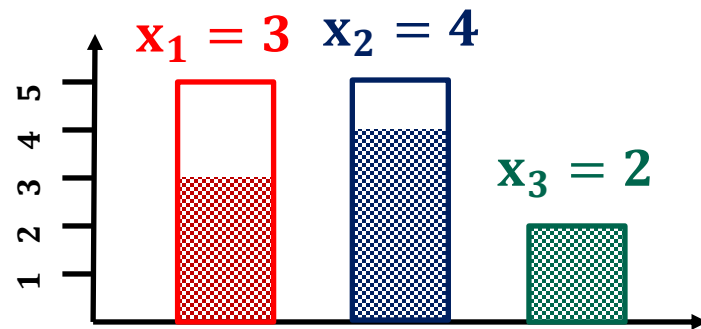
Water Filling Algorithm



□ $C = (10, 9, 5)$ and $d = (5, 5, 2)$



□ List the demands as empty buckets

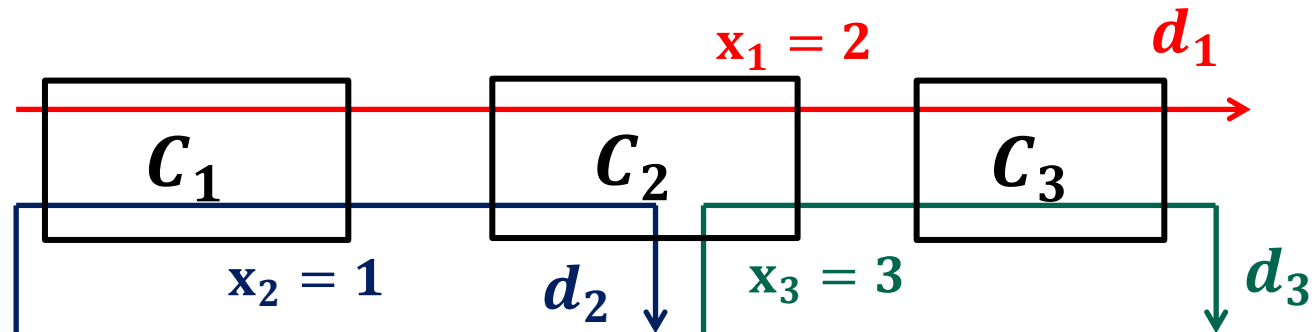


□ "Fill-in water" until one hits bottleneck

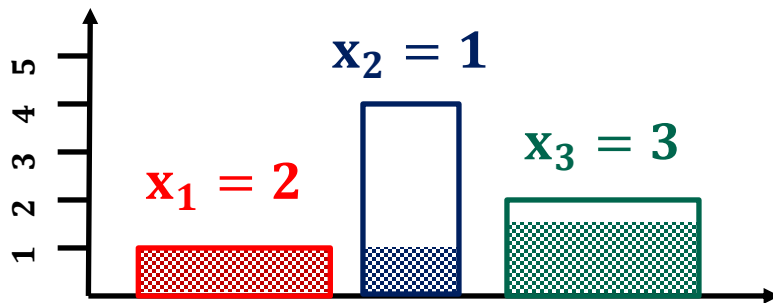
Weighted Max-Min Fair Share

- Extension: weight vector $\phi = (\phi_1, \phi_2 \dots)$
 - no customer receives more than demanded
 - unsatisfied demand split the resource proportional to their weights
- $C = 16, \mathbf{d} = (4, 2, 10, 4), \phi = (2.5, 4, 0.5, 1)$
 - fair share $x_i = \frac{\phi_i}{\sum_j \phi_j} C \Rightarrow \mathbf{x} = (5, 8, 1, 2)$
 - given the demand upper bound $\Rightarrow x_1 = 4, x_2 = 2$
 - distribute the rest 10 proportionally,
(3.33, 6.66), $\Rightarrow x_4 = 4, x_3 = 6$
- How to apply water-filling algorithm?

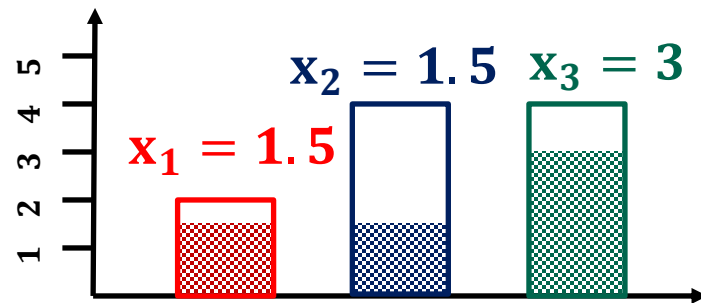
Water Filling Algorithm



□ $C = (3, 6, 7)$, $d = (2, 4, 4)$ and $\phi = (2, 1, 2)$



□ Buckets with width ϕ_i and volume d_i



□ Max-min solution for $\phi = (1, 1, 1)$

Max-Min Fair Implementation

- ❑ How to implement max-min fair flow allocation on packet-based flows?
 - When no centralized information available; distributed mechanisms are needed.
 - Each router needs to implement local scheduling policies (service disciplines).
- ❑ Old solutions: Generalized Process Sharing (GPS) and Weighted Fair Queueing (WFQ)
- ❑ New: Software-Defined Networking (SDN)